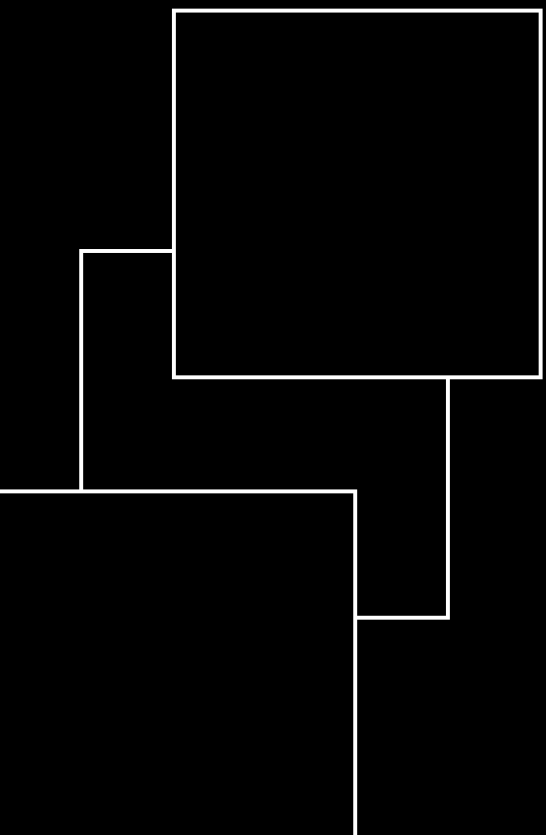
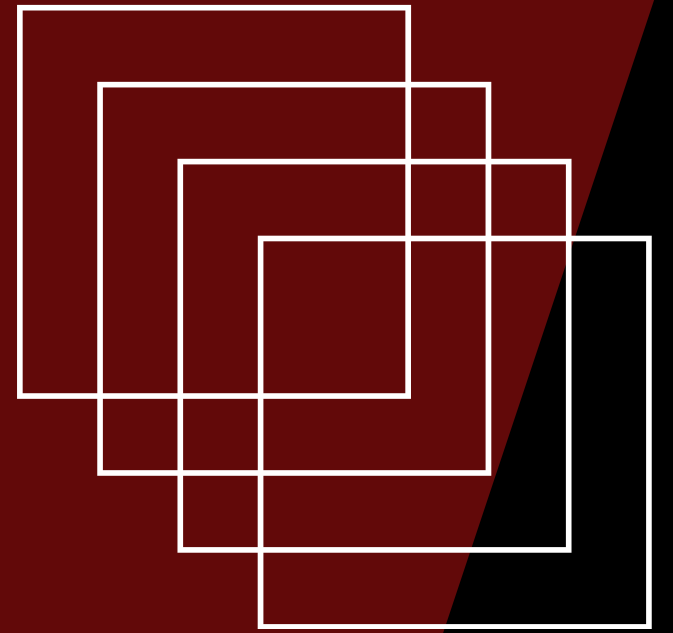
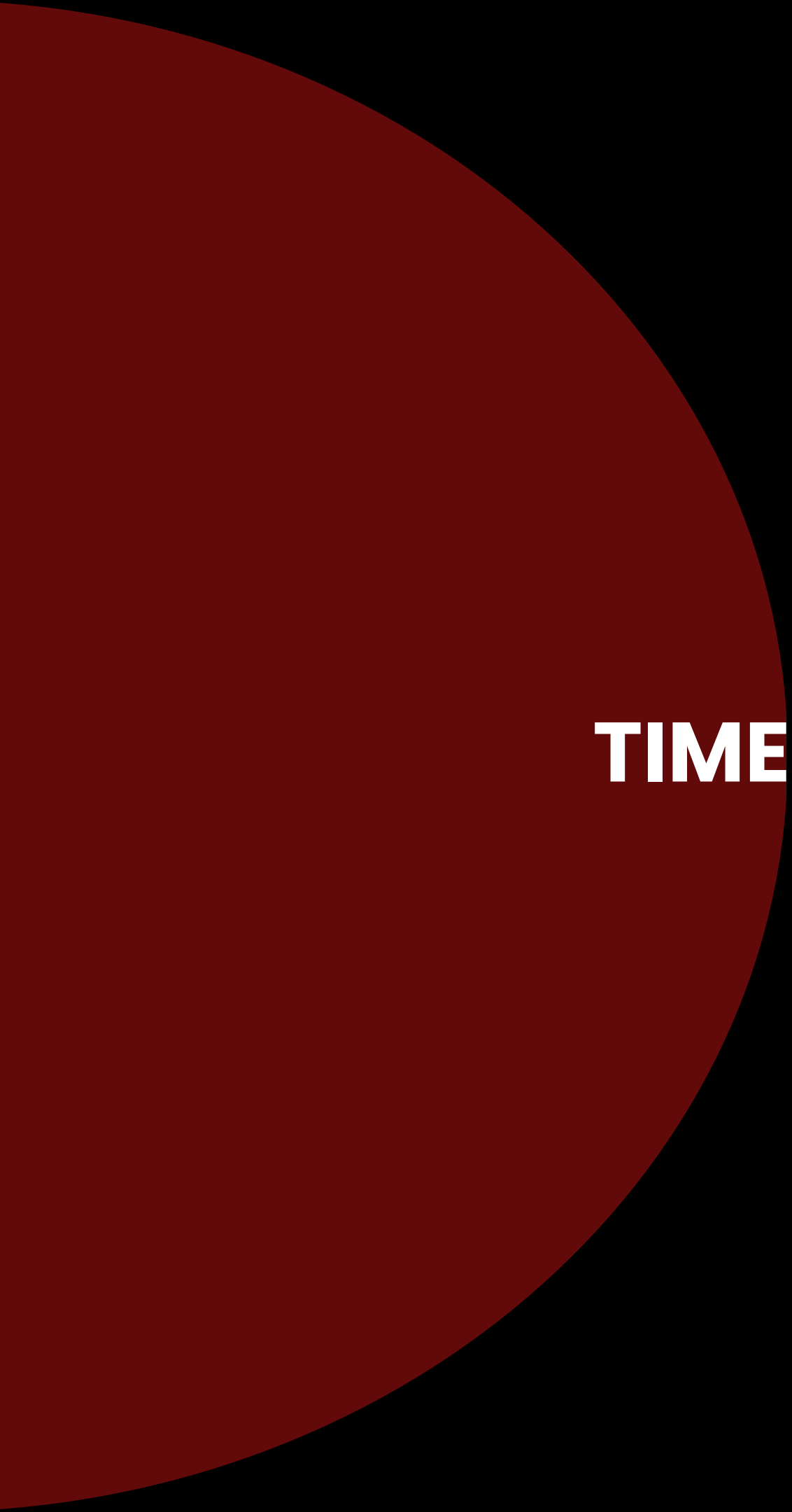


MODERN VISION SYSTEMS

Presented by
Anmay Gupta and Heorhii
Ambarstumov





TIMELINE

1

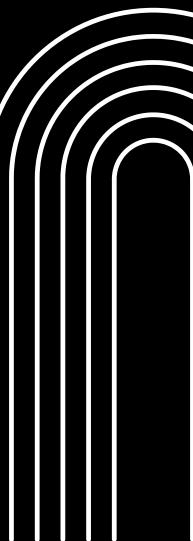
ViTs and Image
Foundations

2

Segmentation
and 3DR

3

Local Models
and Edge ML



PREMODERN VISION

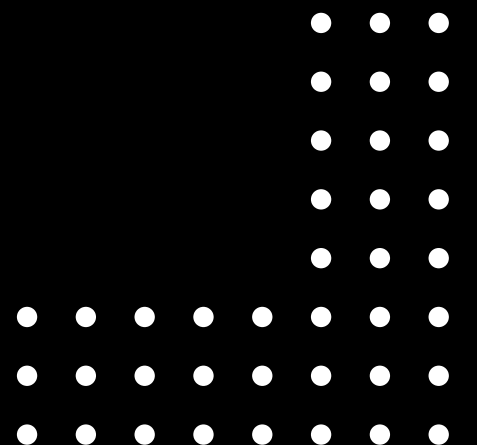
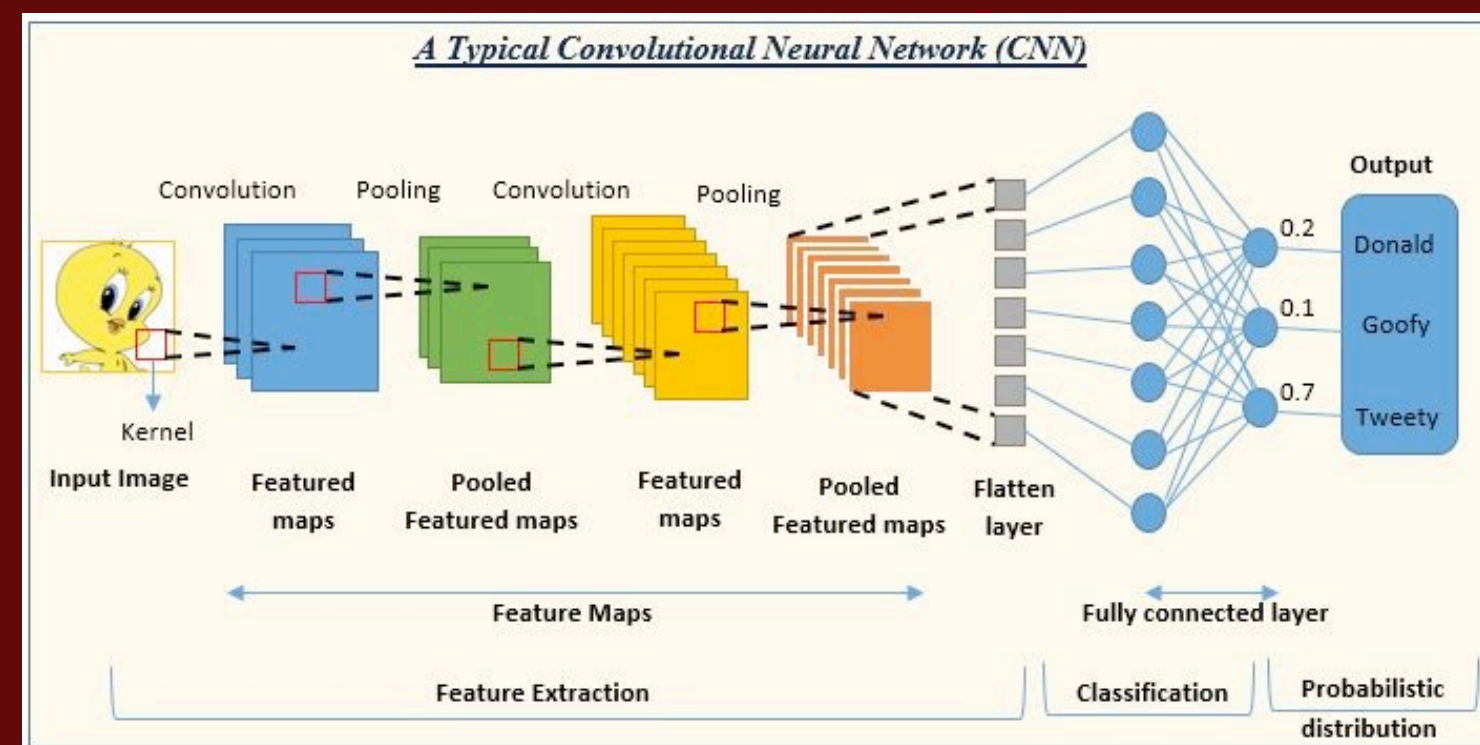
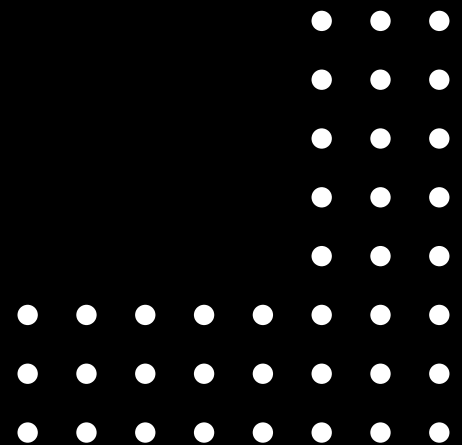
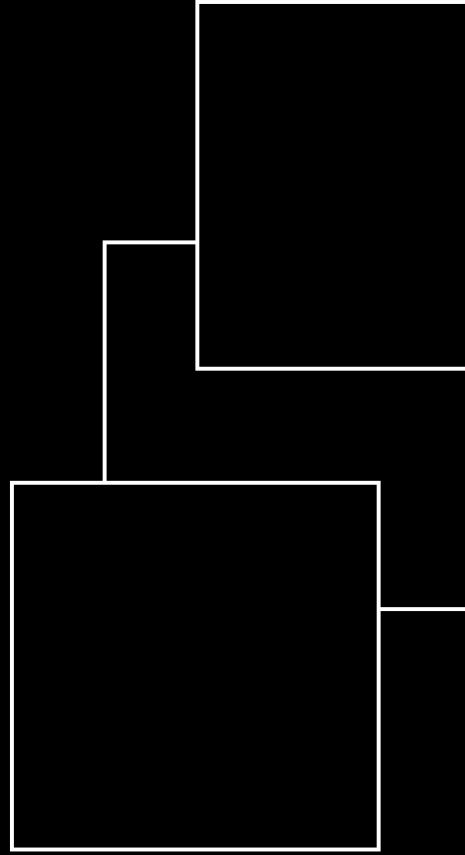


IMAGE FOUNDATION MODELS

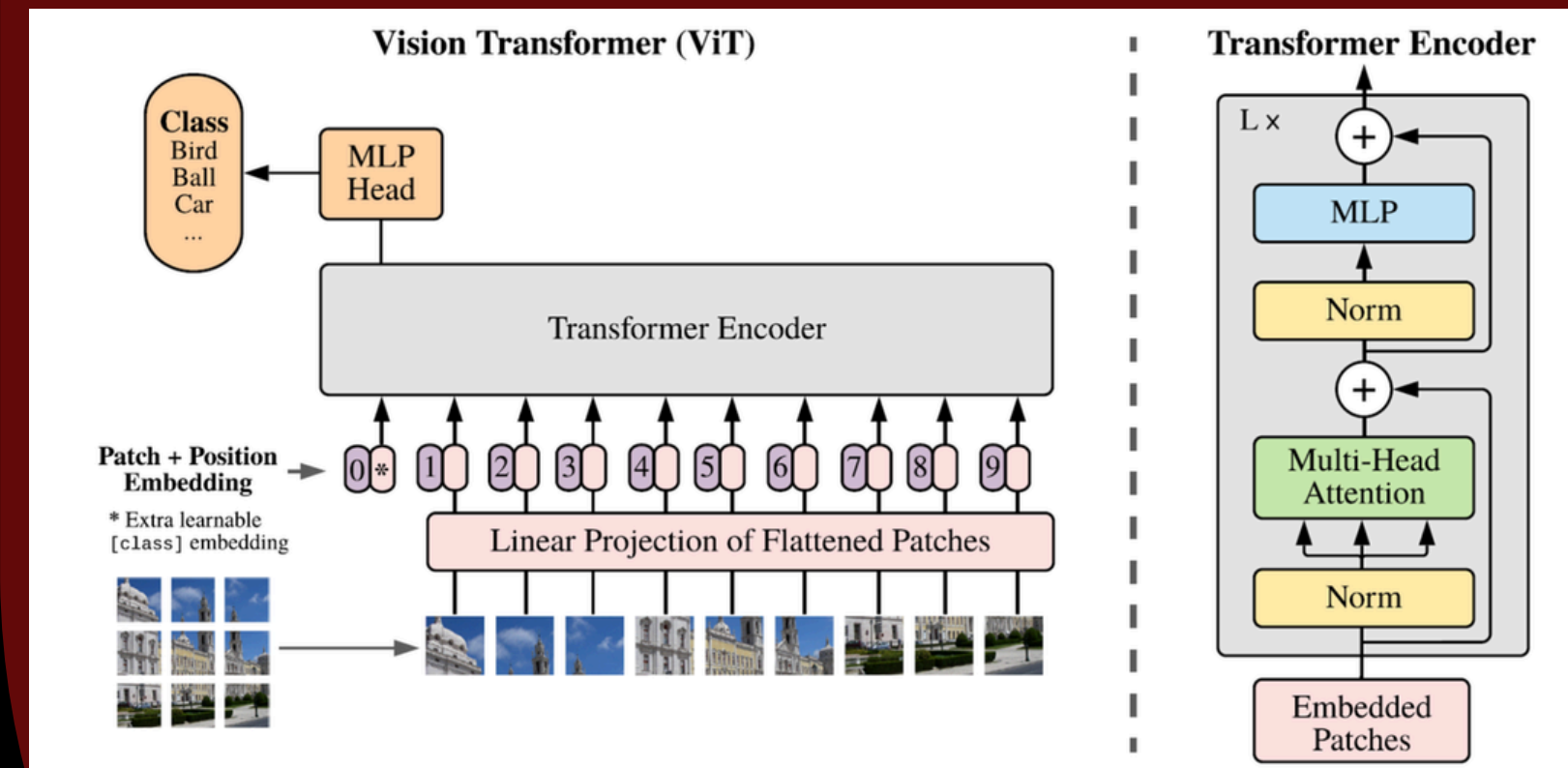


VISION TRANSFORMERS

ViTs presented a way to improve classification accuracy that scales with compute + training data.

By cutting images into patches and adding an additional classification token, ViTs formed the basis of future experimentation into modern CV.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k



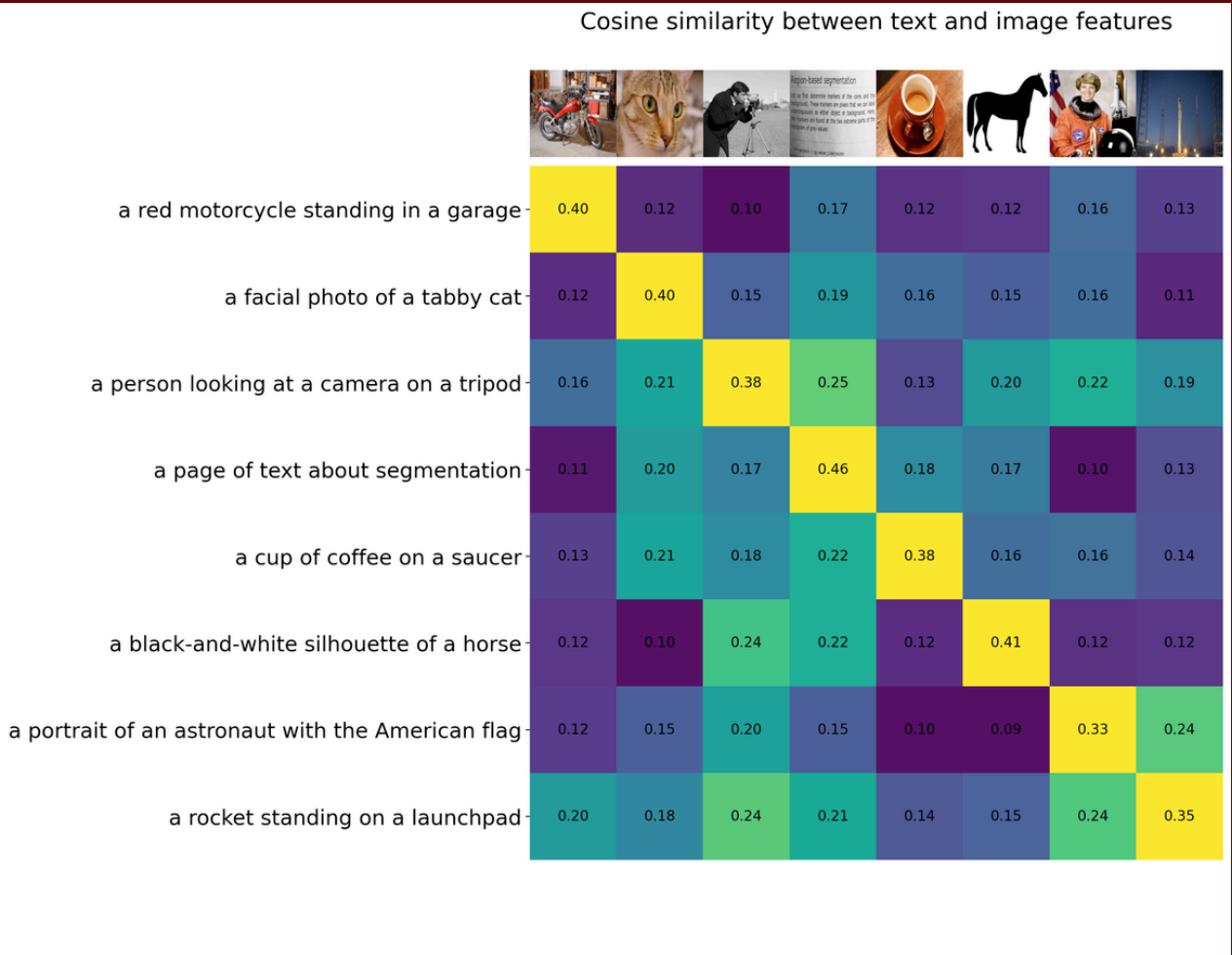
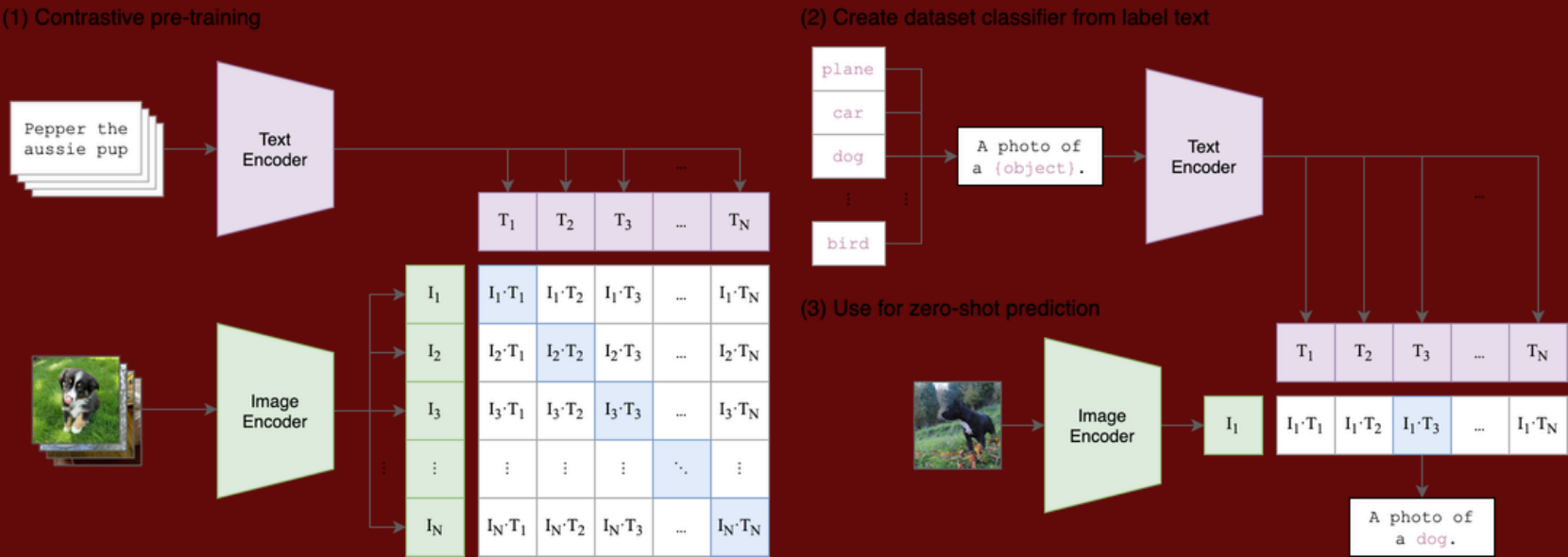
CLIP

Showed that ViTs can be trained on web-scale and web-quality data and reach high performance.

Enabled dataset-level generalization, beyond previous category-level attempts.

Pioneered a new way of training using contrastive learning to improve embedding quality.

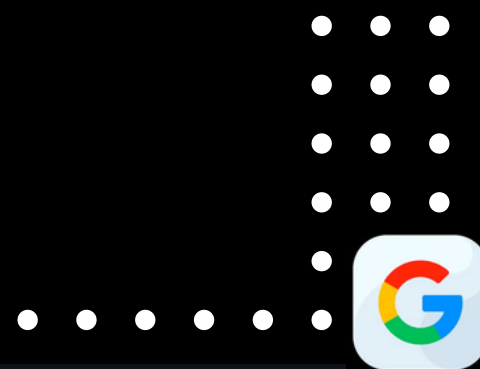
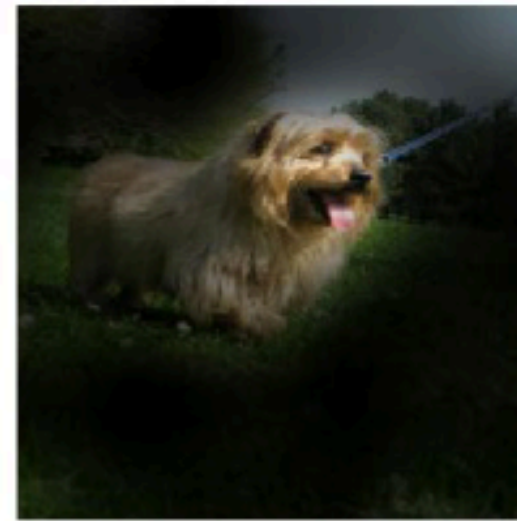
	aYahoo	ImageNet	SUN
Visual N-Grams	72.4	11.5	23.0
CLIP	98.4	76.2	58.5



Input

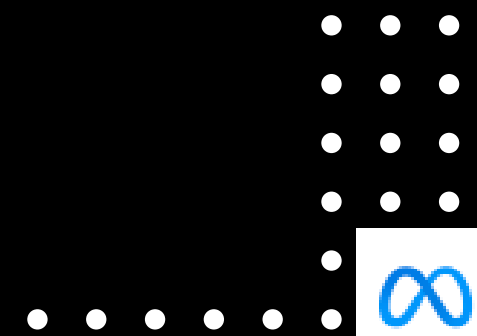
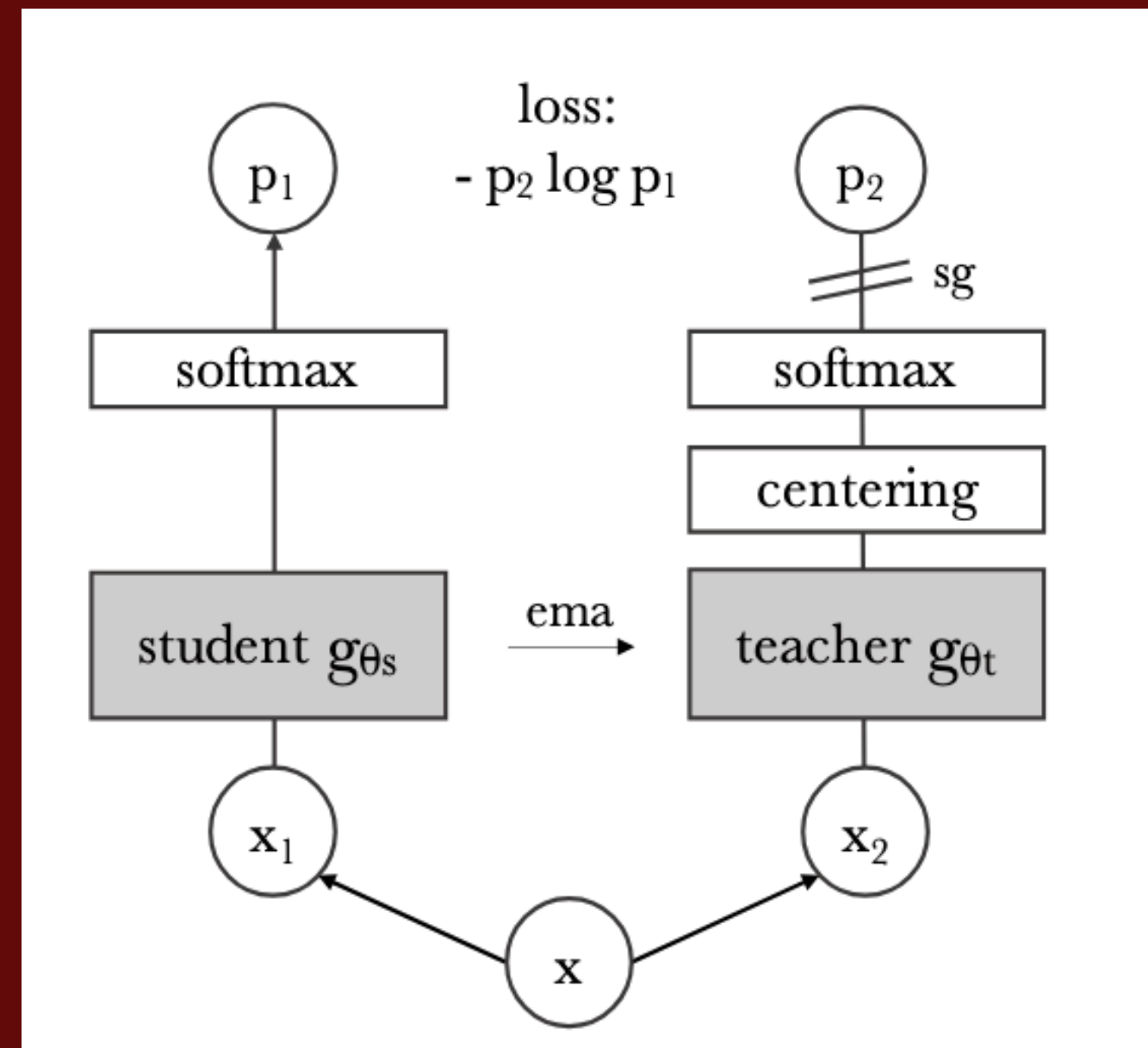


Attention



DINO

FOUNDATION MODELS



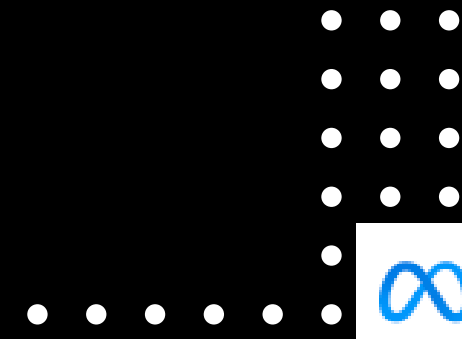
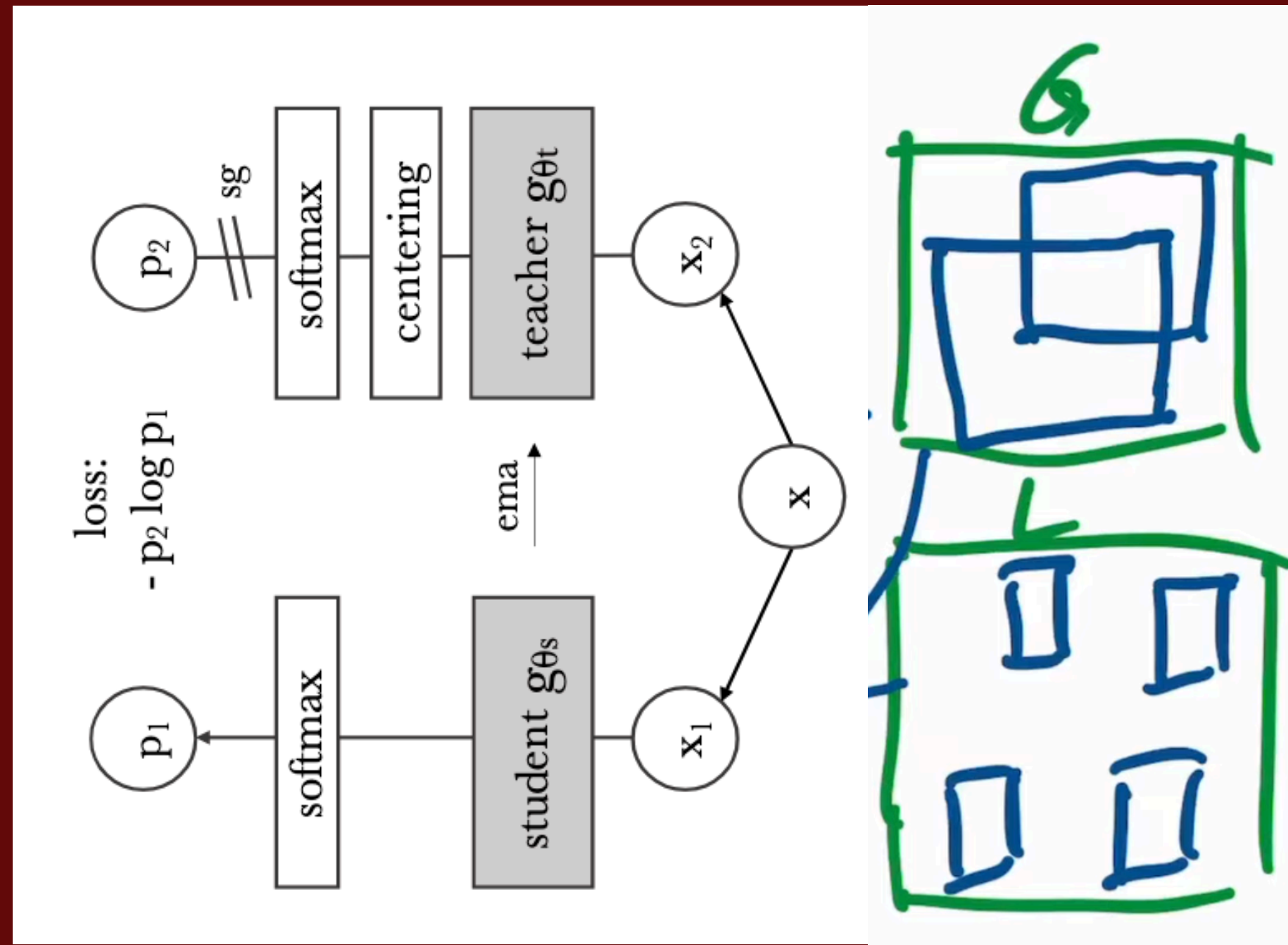
Star

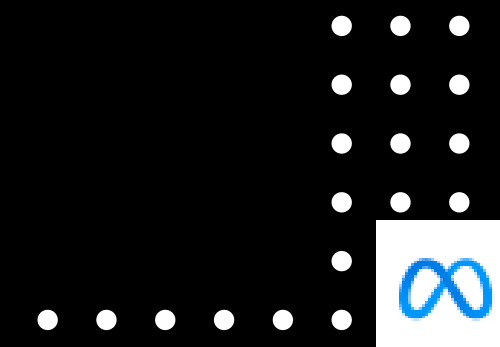
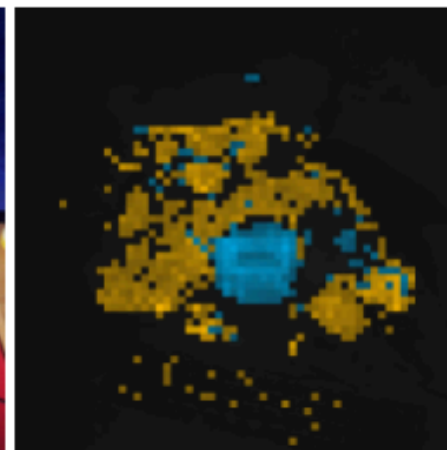
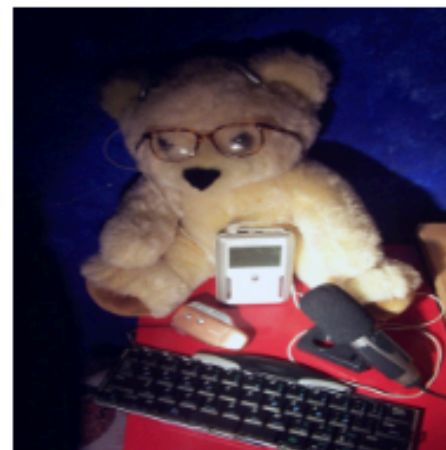
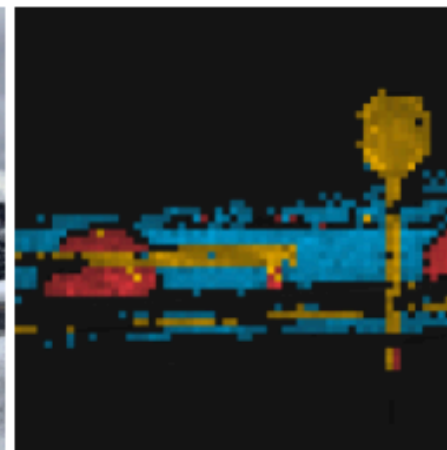
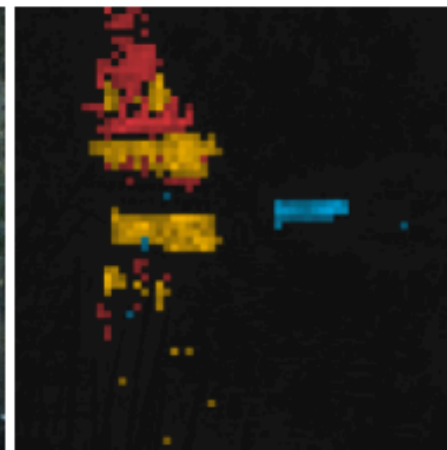
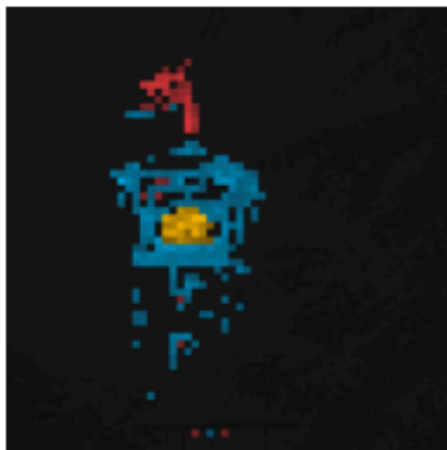
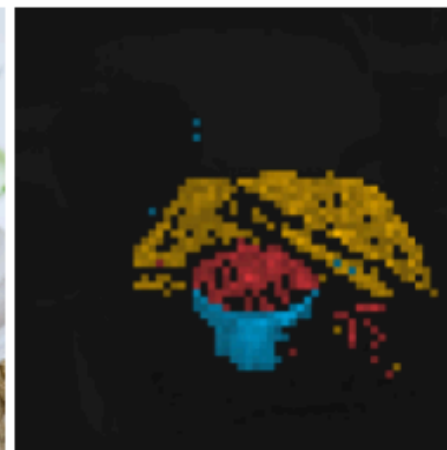
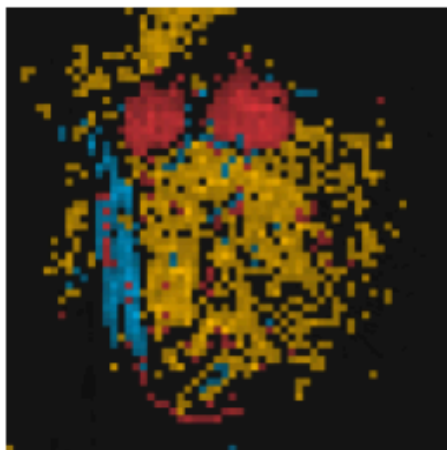
7.4k



DINO

FOUNDATION MODELS

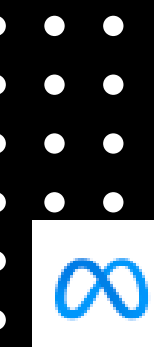
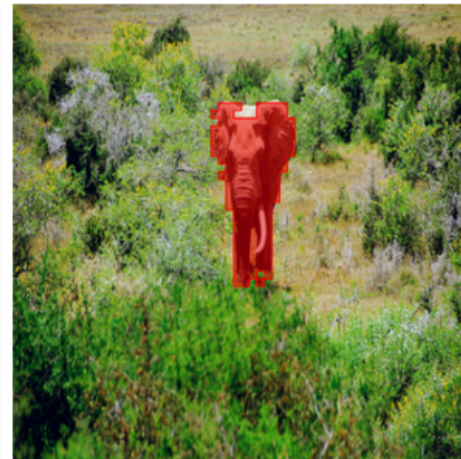
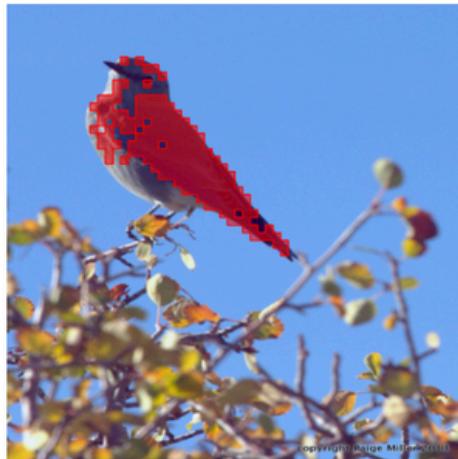


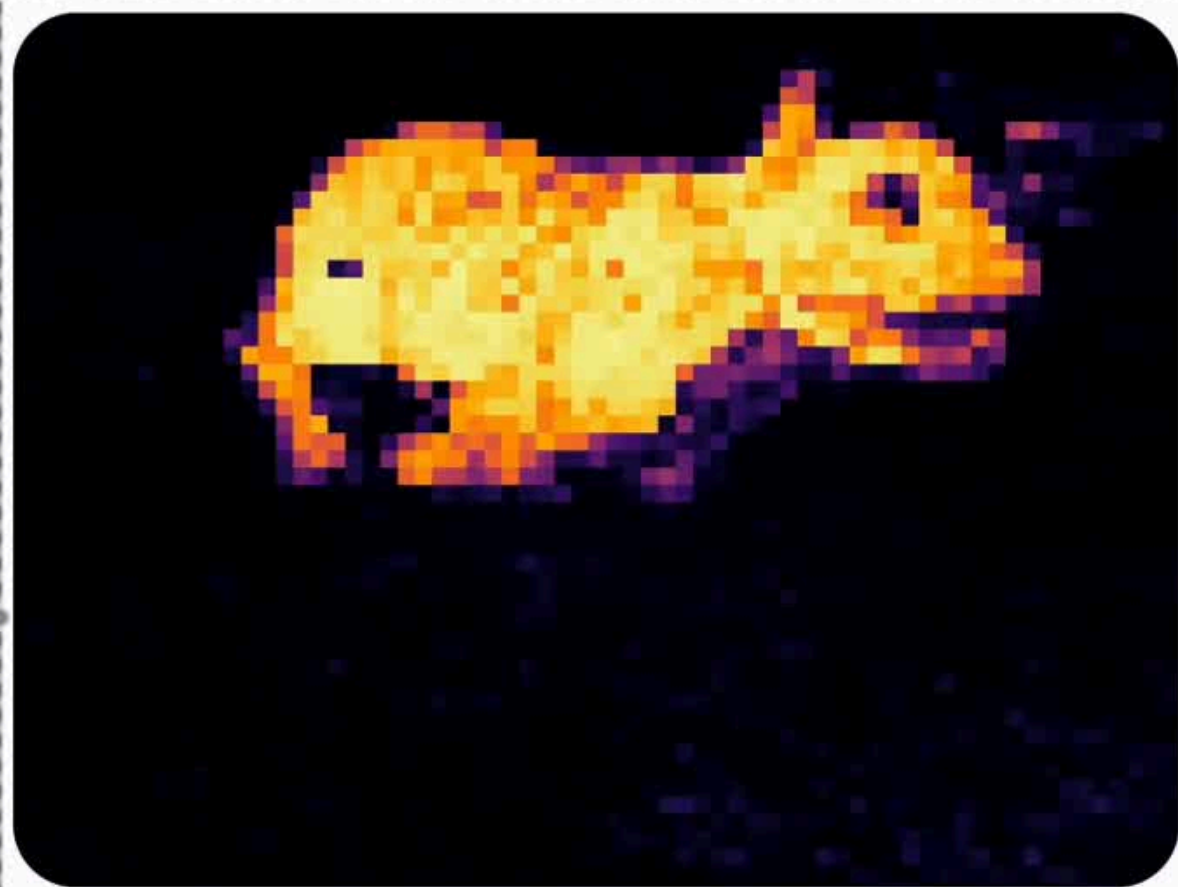


Supervised



DINO





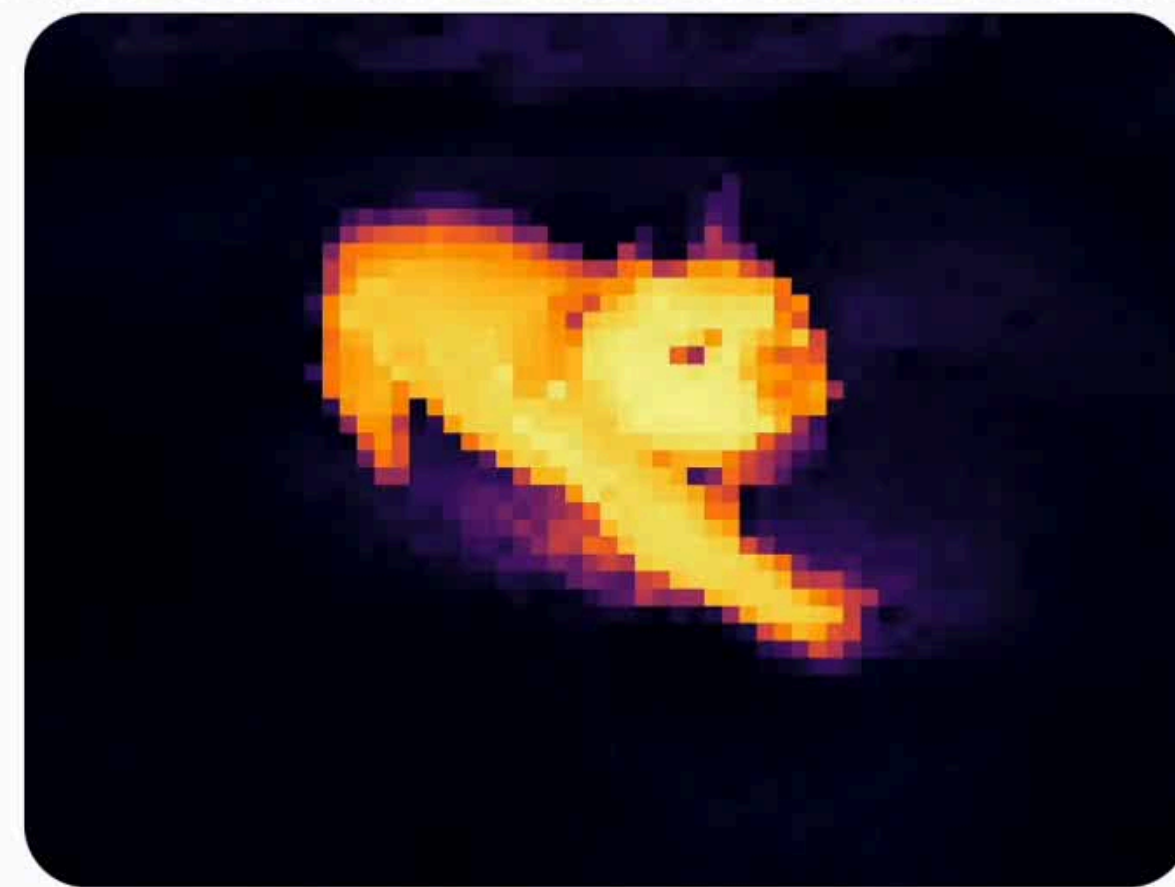
DINO

Initial research proof-of-concept, with 80M-parameter models trained on 1M images.



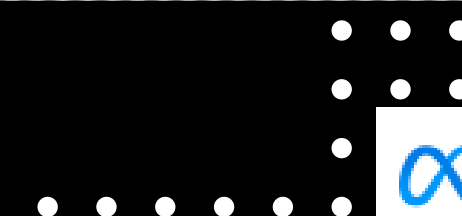
DINOv2

First successful scaling of a SSL algorithm. 1B-parameter models trained on 142M images.



DINOv3

An order of magnitude larger training compared to v2, with particular focus on dense features.



CLASSIFICATION → SEGMENTATION

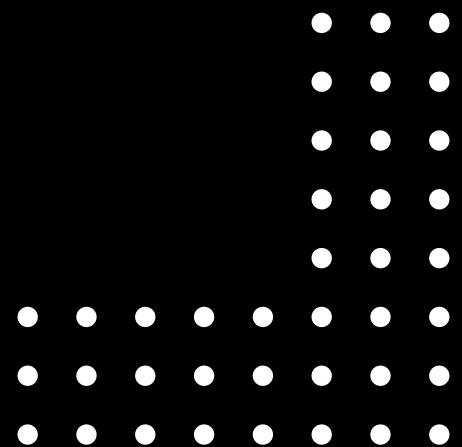
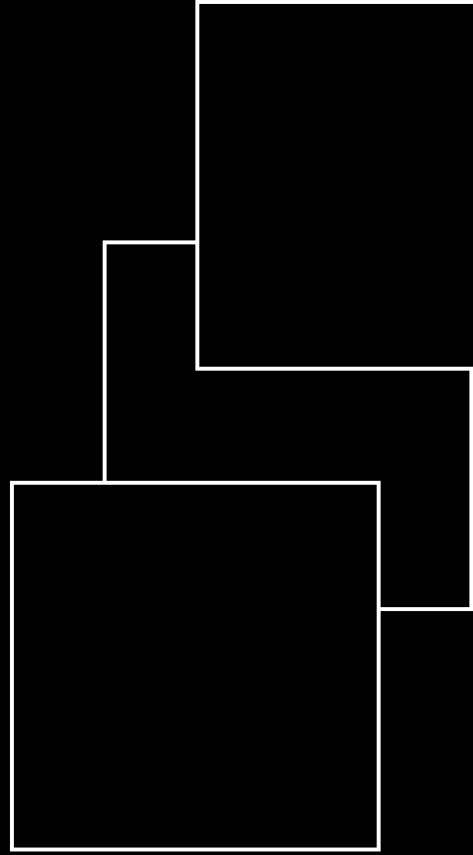
PROBLEM 01

Rather than relying on bounding boxes, can we use transformers to pick up tight segmentations on various different kinds of visual data?

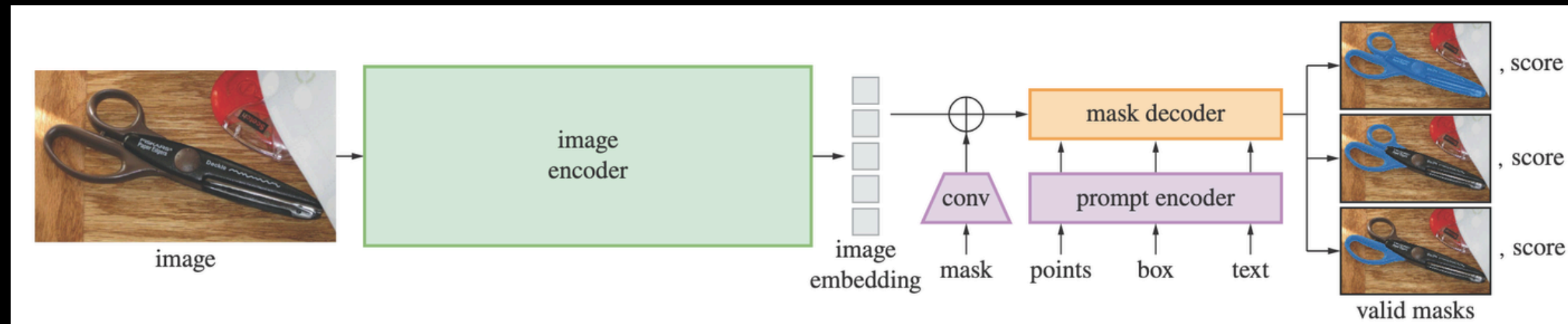
PROBLEM 02

What other information can we get out of ViTs trained using DINO?

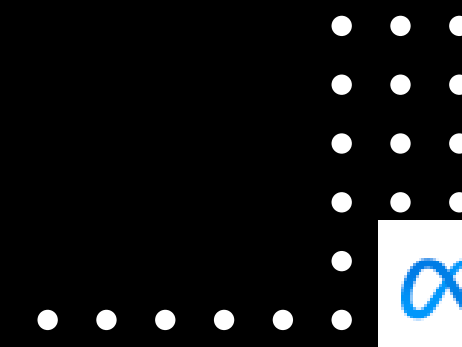
SEGMENTATION + 3D RECONSTRUCTION



SAM

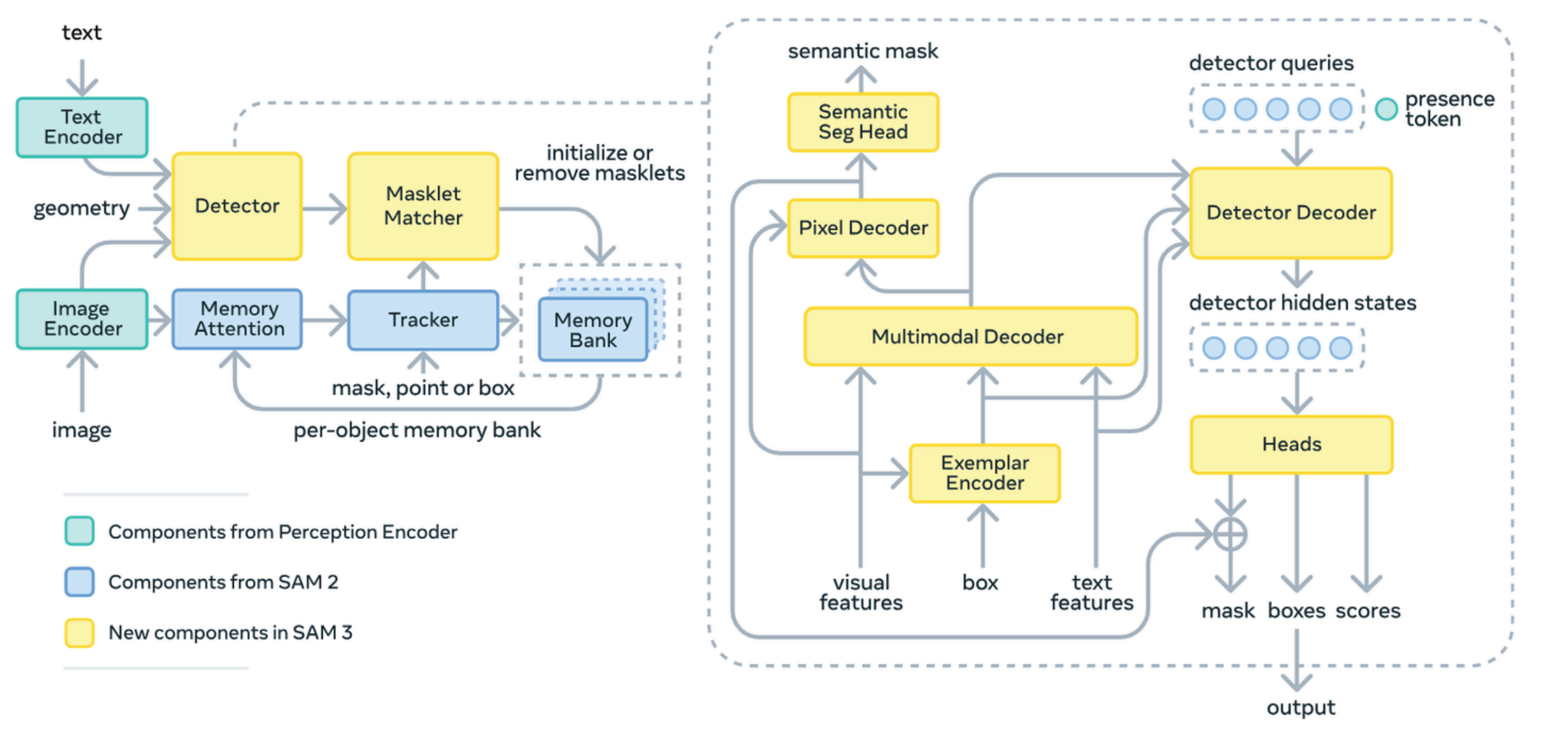


Using 1.2M human-generated masks and eventually off of self-annotated data, a mask decoder on a ViT could extract full masks

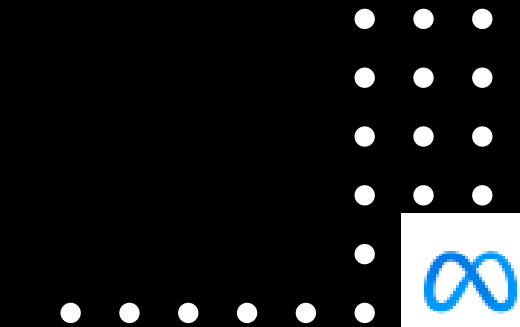




SAM3



Expanding to allow for promptable segmentation by feeding text features as well, allows for LLM-based agents to extract segmentations





SAM3



Figure 6 Example video (top) and images (bottom) from SA-Co with annotated phrases and instance masks/IDs.

see demo at <https://aidemos.meta.com/segment-anything/gallery>

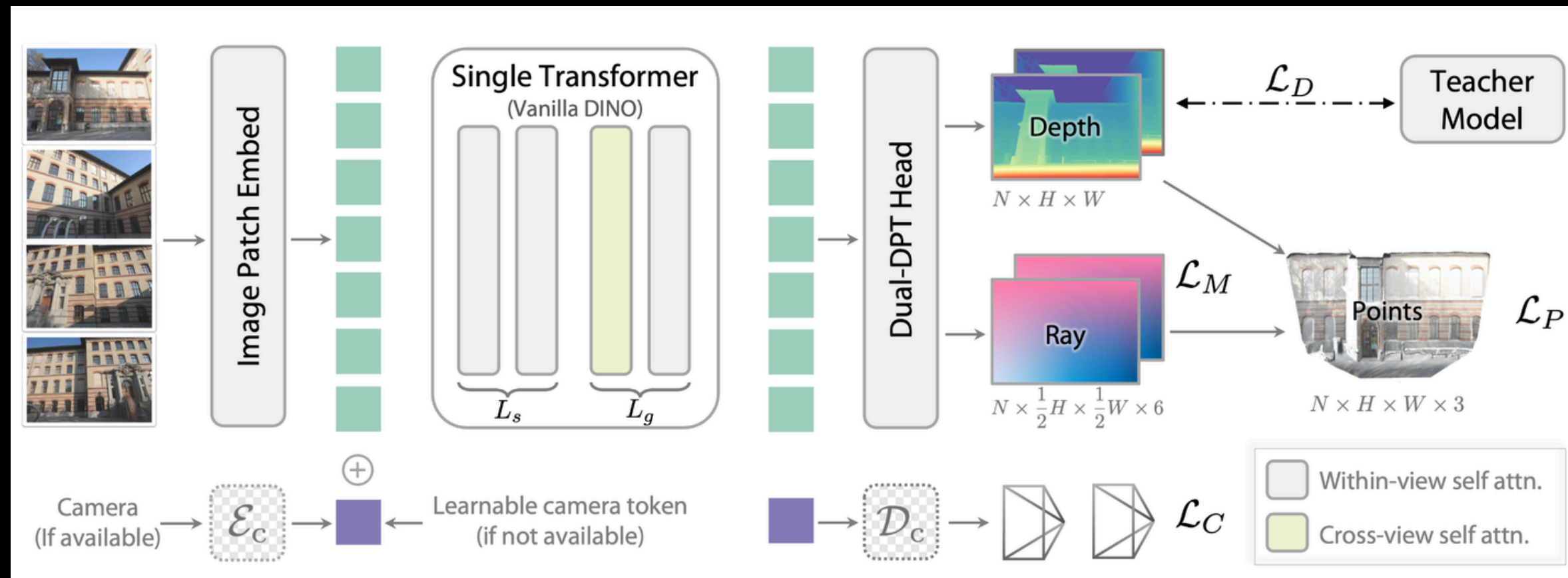


Star 7.7k

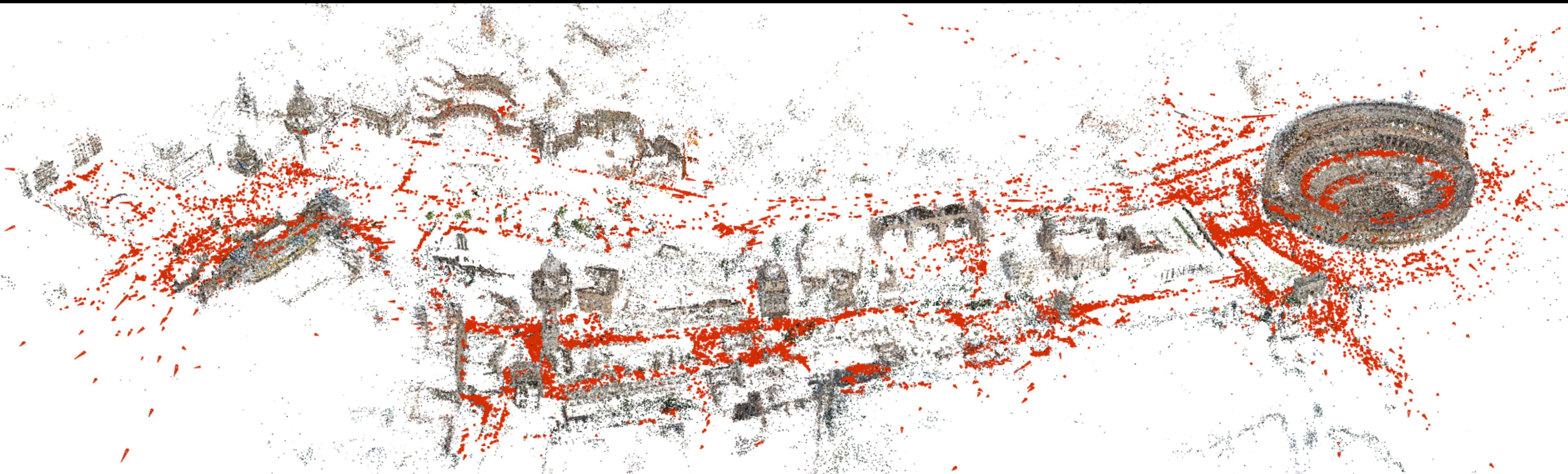


3D RECONSTRUCTION

Depth-Anything

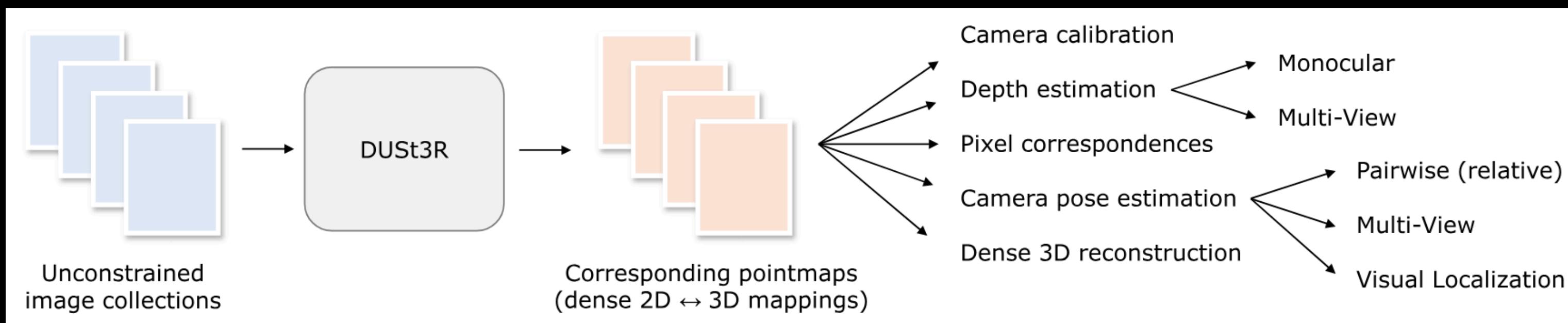
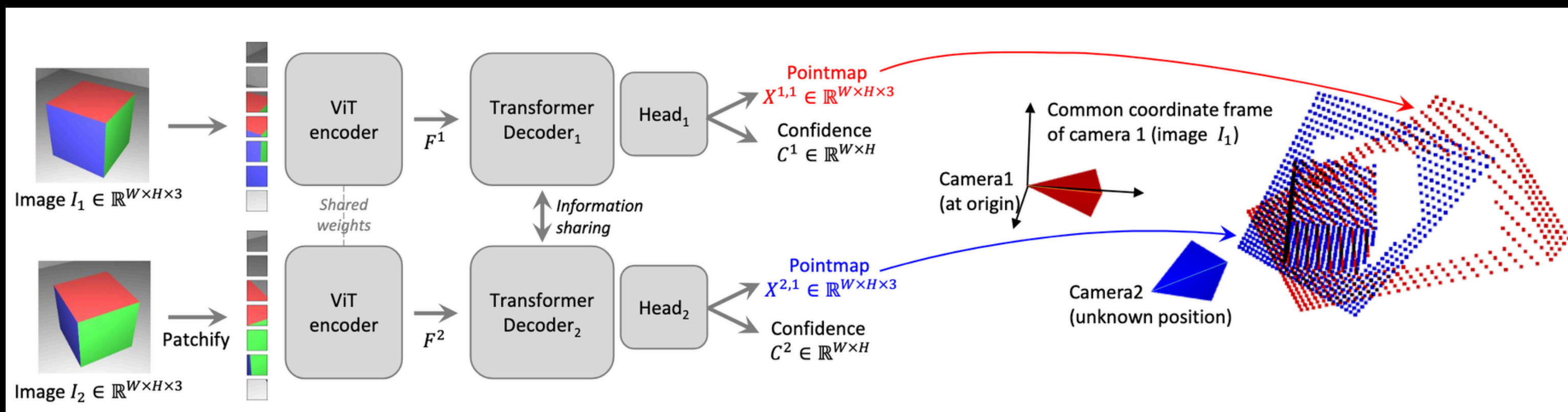


Building Rome in a Day

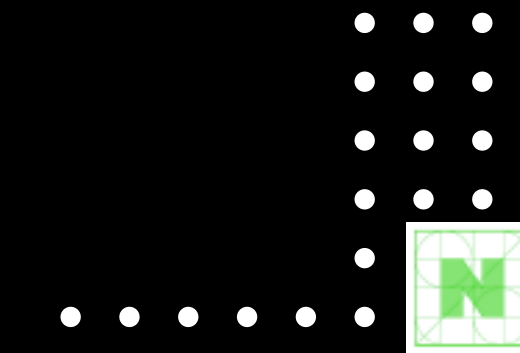
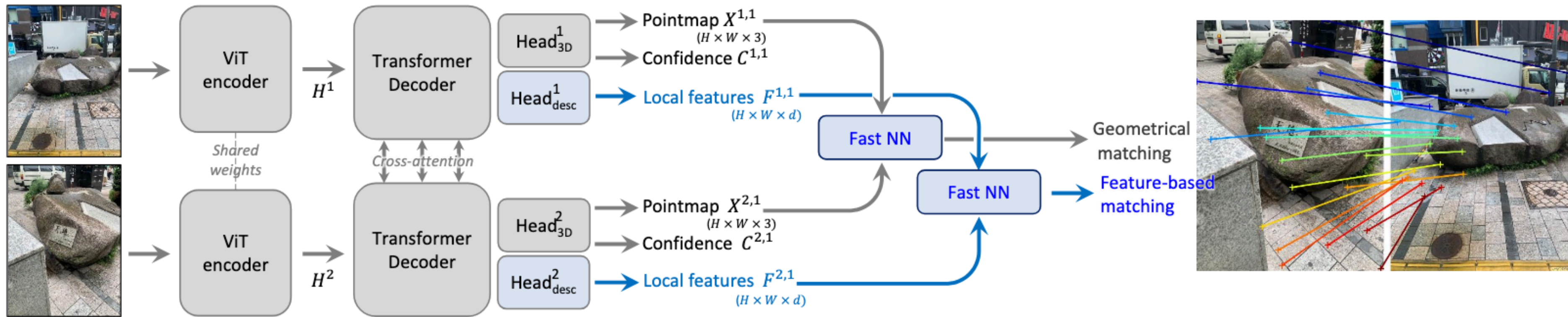


Problem known as Structure from Motion (SfM) allows for 3D reproduction of camera poses from standard RGB-D images pulled from Flickr. Researchers in 2009 from UW used this to build a pointmap of Rome, but it took 24 hours and a platoon of GPUs.

DUST3R



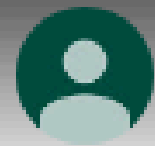
MASt3R



3D Gaussian Splatting



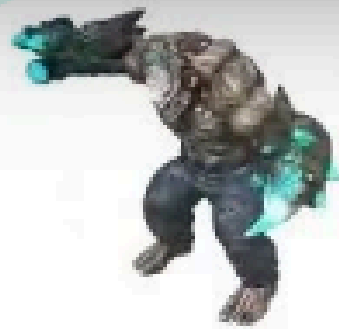
4D Gaussian Splatting



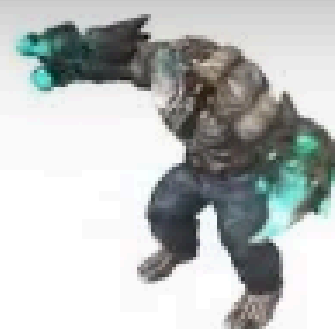
[CVPR2024]4D Gaussian Splatting for Real-Time Dynamic Scene Rendering



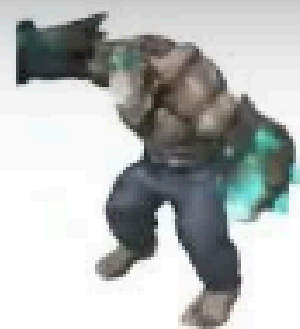
Copy link



Ours (82FPS)



HexPlane (4FPS)



TiNeuVox (1FPS)



Ours (82FPS)



HexPlane (4FPS)



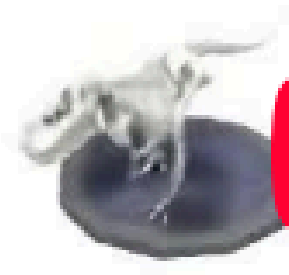
TiNeuVox (1FPS)



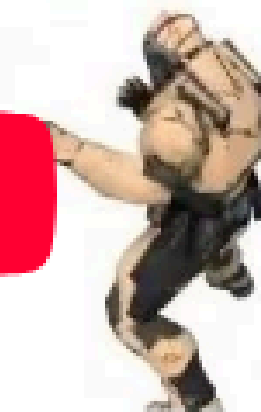
Ours (82FPS)



HexPlane (4FPS)



TiNeuVox (1FPS)



Ours (82FPS)



HexPlane (4FPS)



TiNeuVox (1FPS)



Watch on YouTube



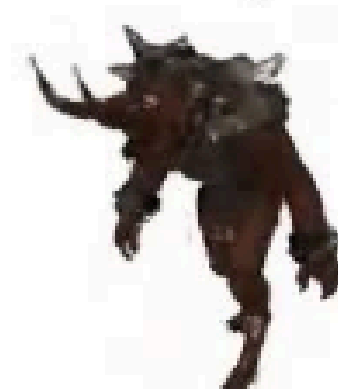
HexPlane (4FPS)



TiNeuVox (1FPS)



Ours (82FPS)

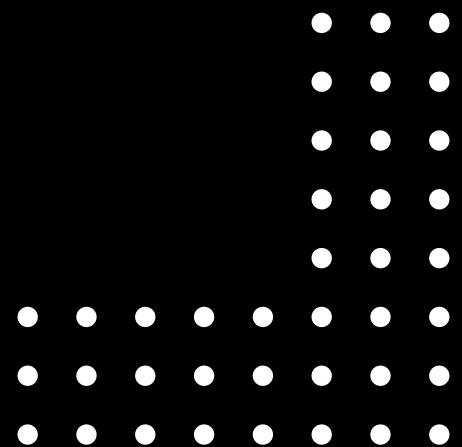
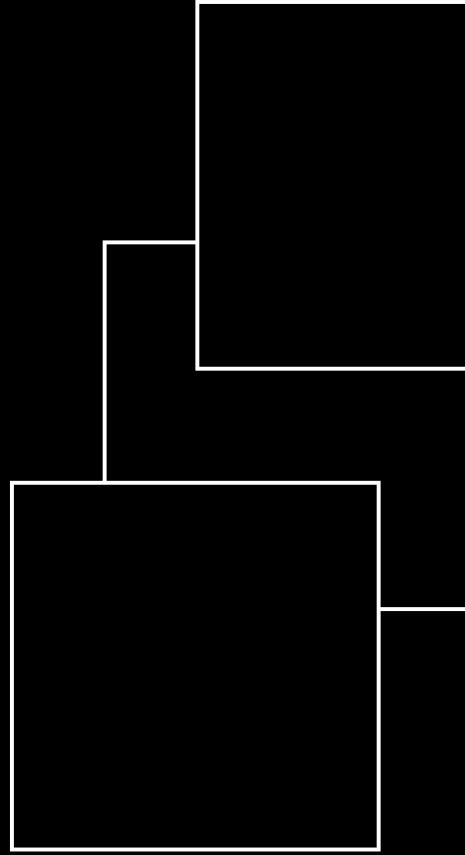


HexPlane (4FPS)



TiNeuVox (1FPS)

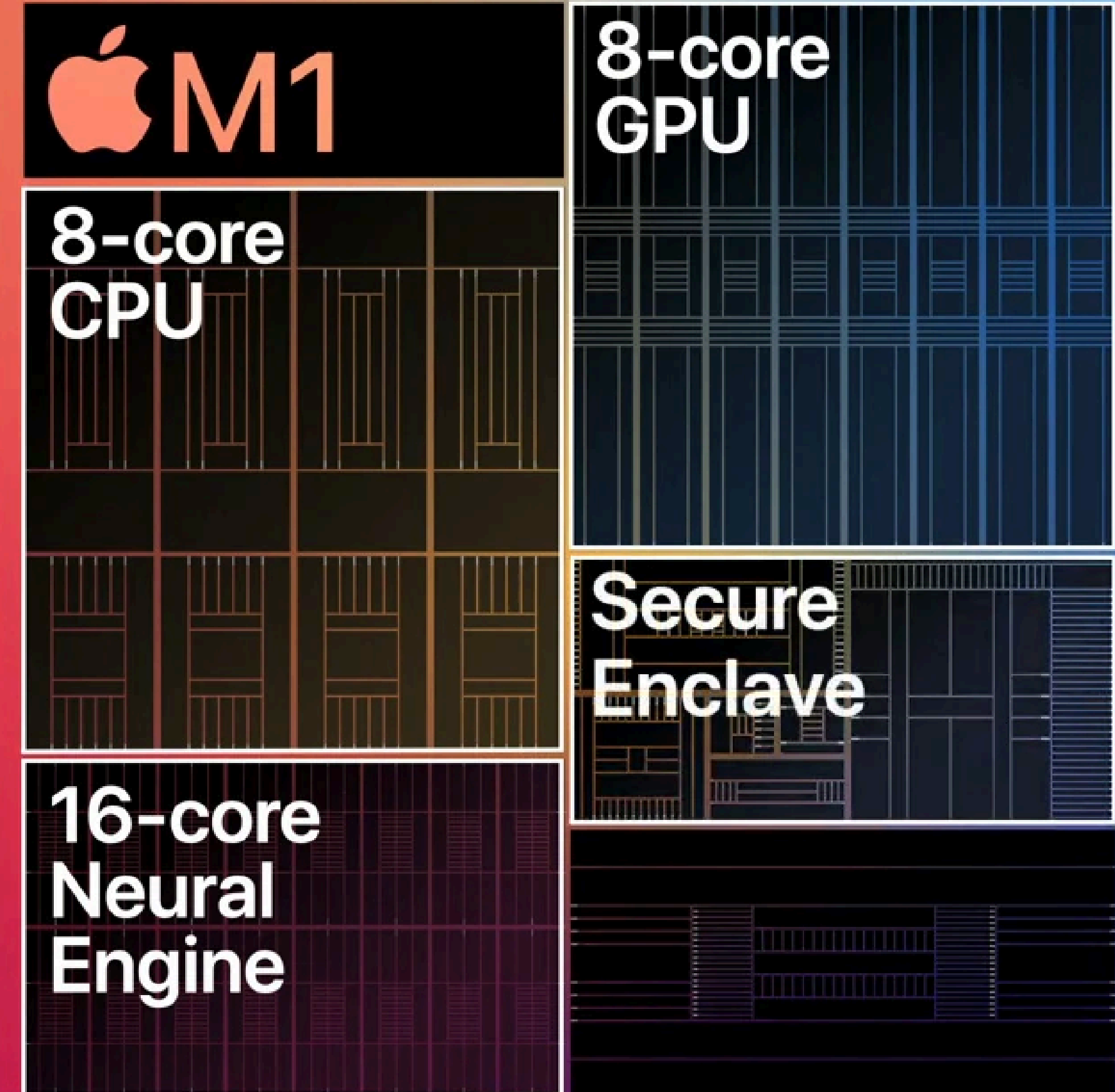
INFERENCE + CROSS- COMPABILITY



NEURAL ACCELERATORS VS GPUS

- GPUs are for general purpose linalg bash
- GPUs have flexible data paths
- TPUs have more streamlined data paths optimized for neural nets
- Less operations, but win on speed
- Therefore better efficiency (in particular, battery efficiency)

You can optimize even further for specific architectures like transformers.



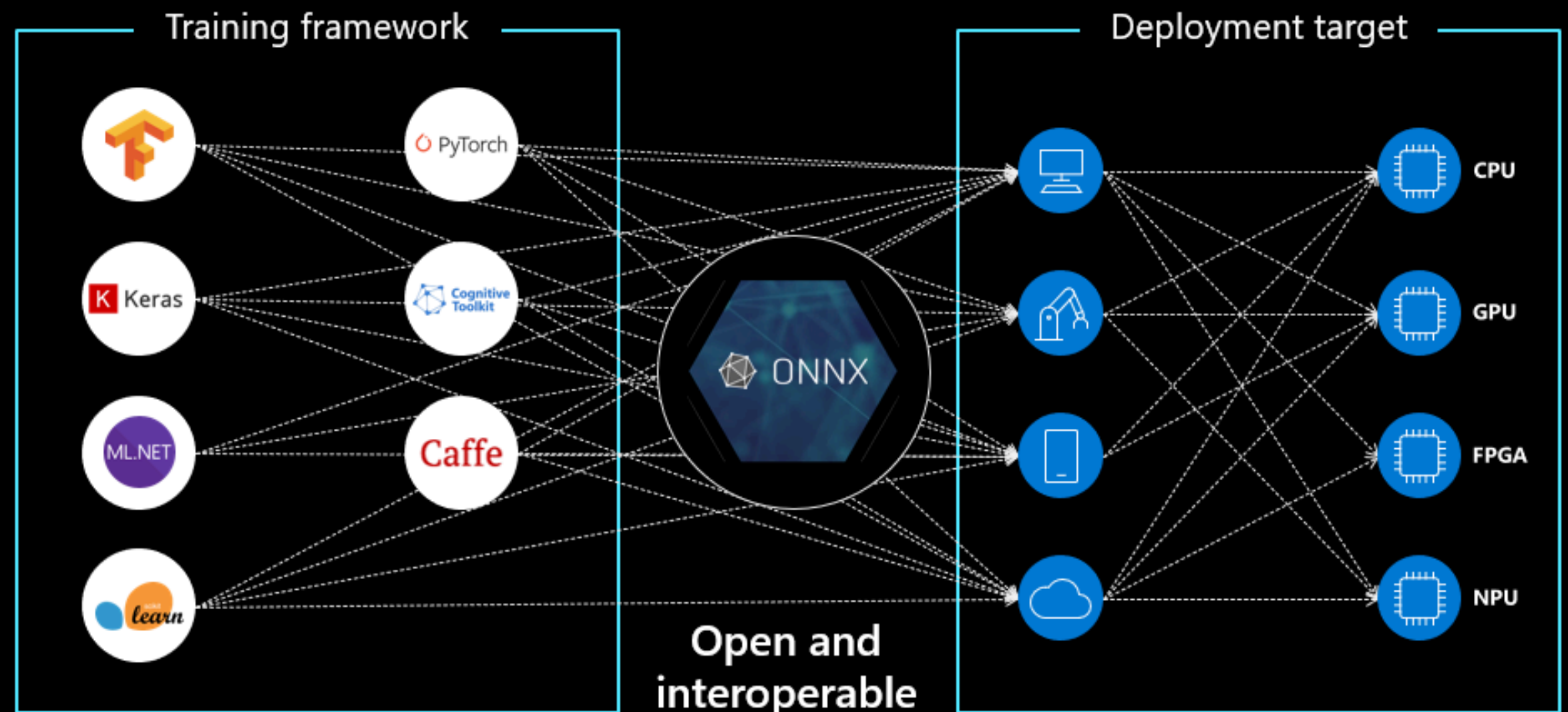
GENERAL OPTIMIZATION STUFF

- Kernel fusion: Combine adjacent operations (e.g., Conv + Bias + ReLU) into a single GPU kernel
- Mixed precision (FP16 / BF16)
- Memory layout optimization: Use contiguous tensor layouts (throwback to warps)
- Process larger batches or using tiling strategies that keep data in shared memory longer
- Quantization: integer arithmetic when supported efficiently by the GPU backend



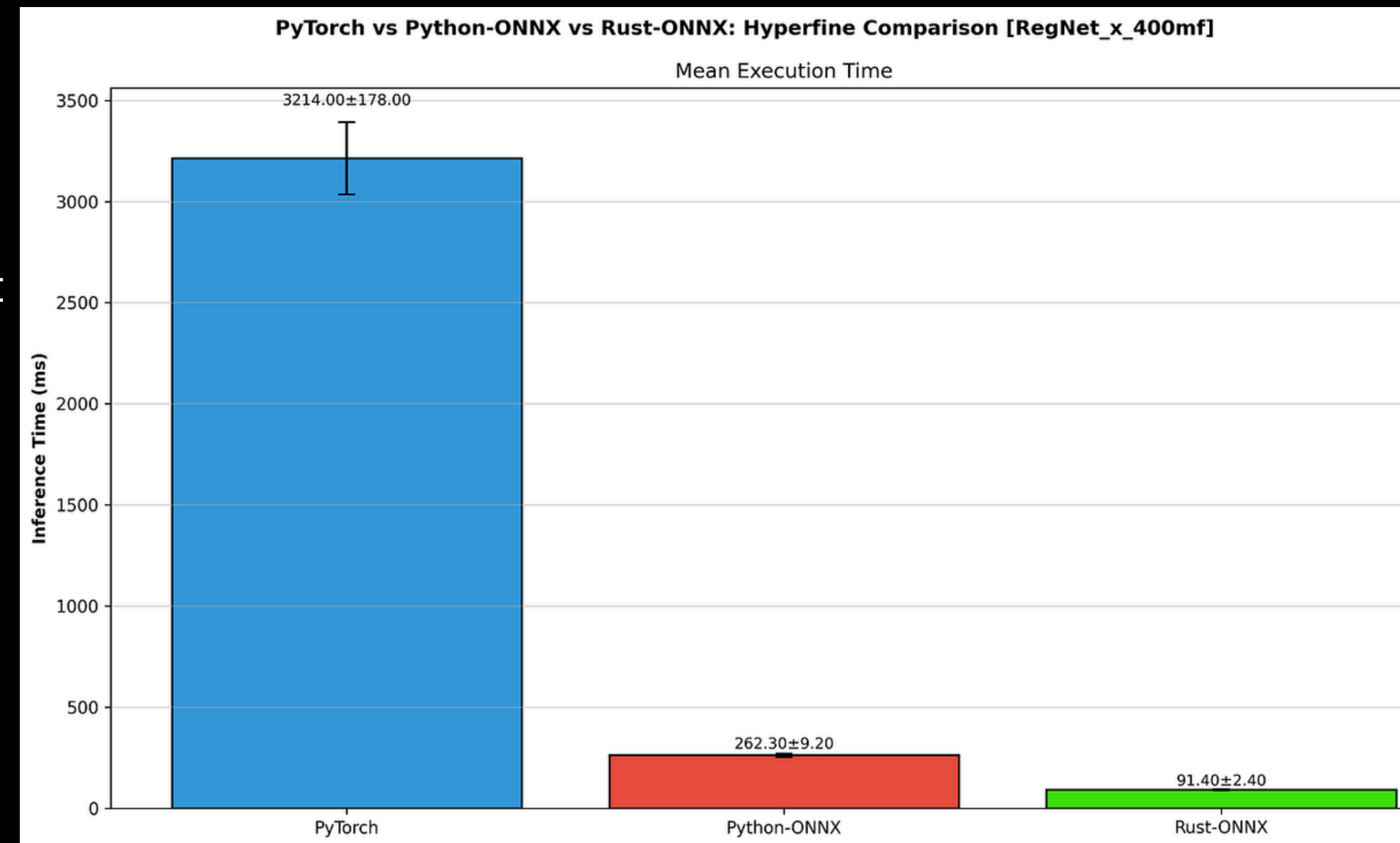
ONNX

- Each library like PyTorch, TensorFlow etc. represents model in its own way
- We can represent an NN in a library-agnostic way
- Think LLVM but for NNs



ONNX

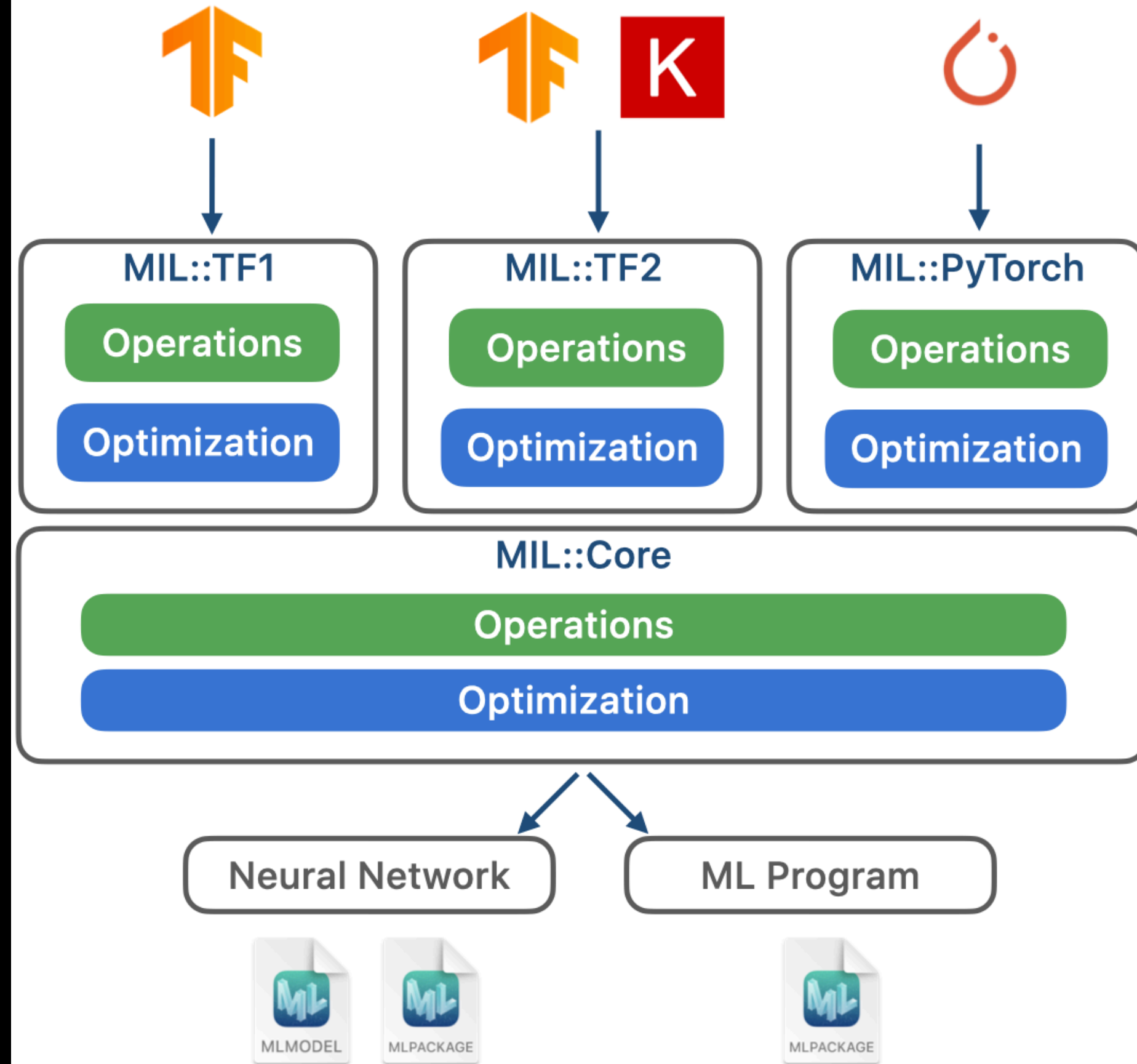
- Each library like PyTorch, TensorFlow etc. represents model in its own way
- We can represent an NN in a library-agnostic way
- Think LLVM but for NNs
- It's fast!
- There are analogs like OpenVINO (Intel), Tensorflow Lite (Android), Core ML (Apple)
- ONNX is the most general runtime that's not heavily optimized for specific hardware



COREML

Apple's own runtime/optimizer

- Converts models into IR, compiler-style optimization
 - graph rewrite
 - subgraph splitting for CPU, GPU, Neural Engine etc.
 - hardware-specific kernels/ops
- Model is hardware-aware and scheduled across CPU, GPU, and Apple Neural Engine



MOBILENET

Can also make improvements to models by using token mixing or other benchmarks to improve performance

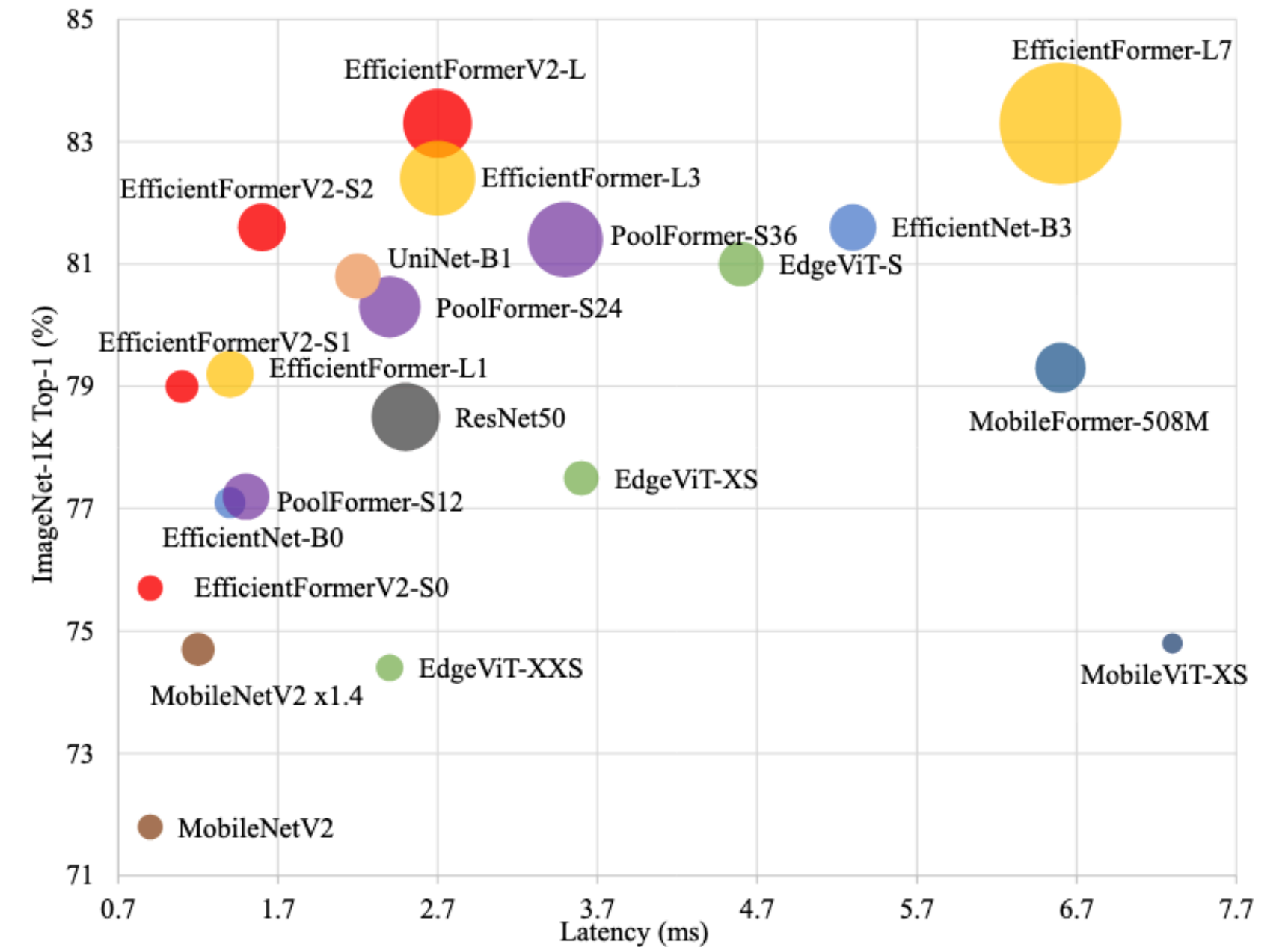
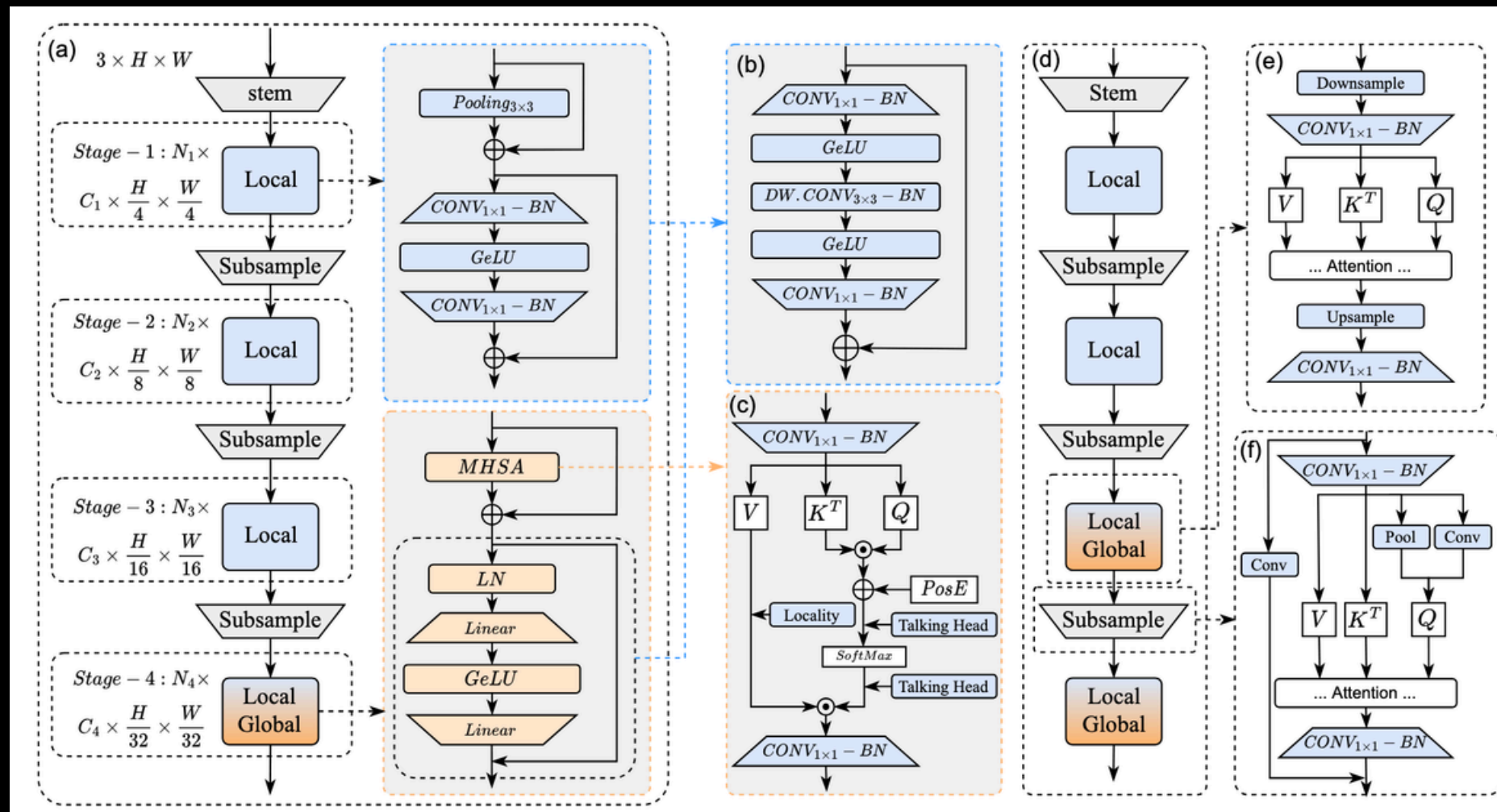
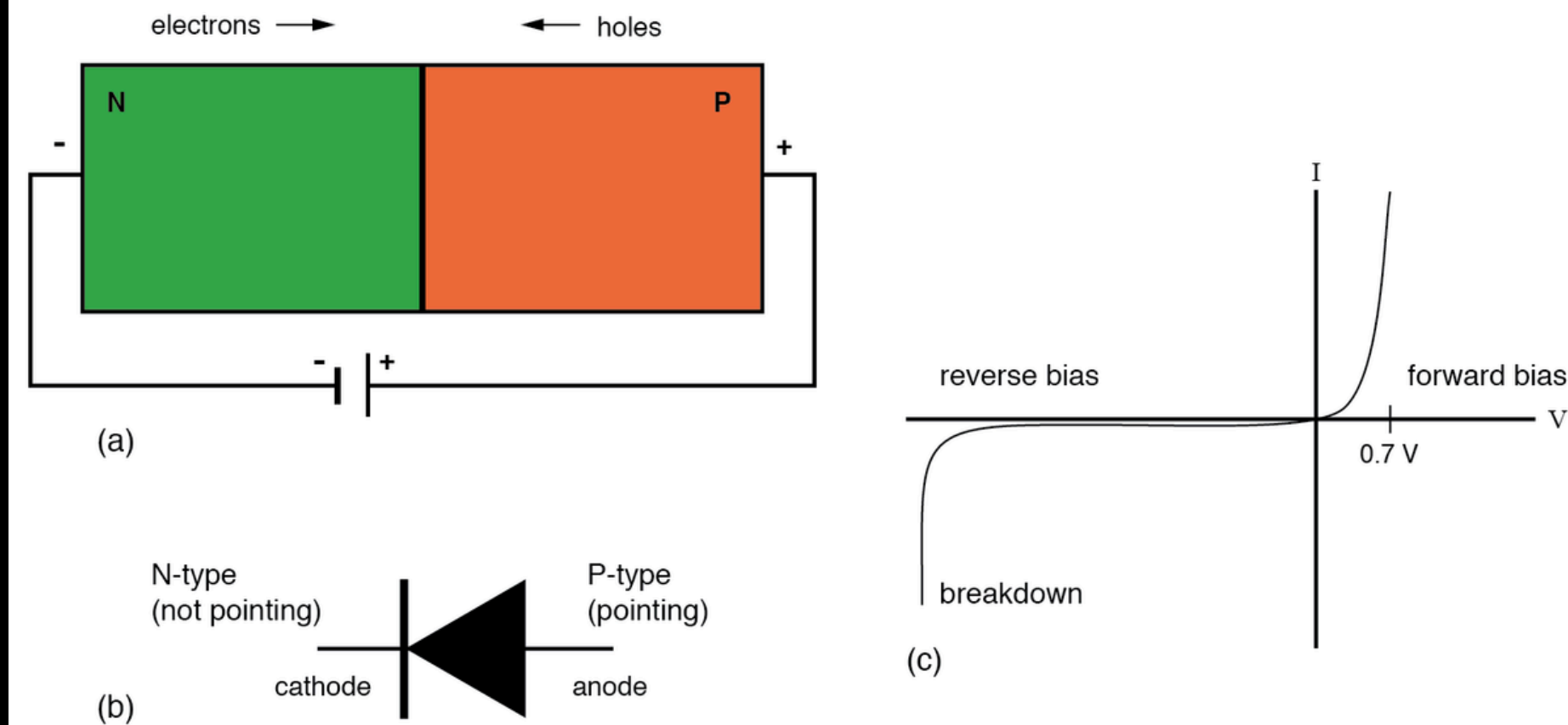


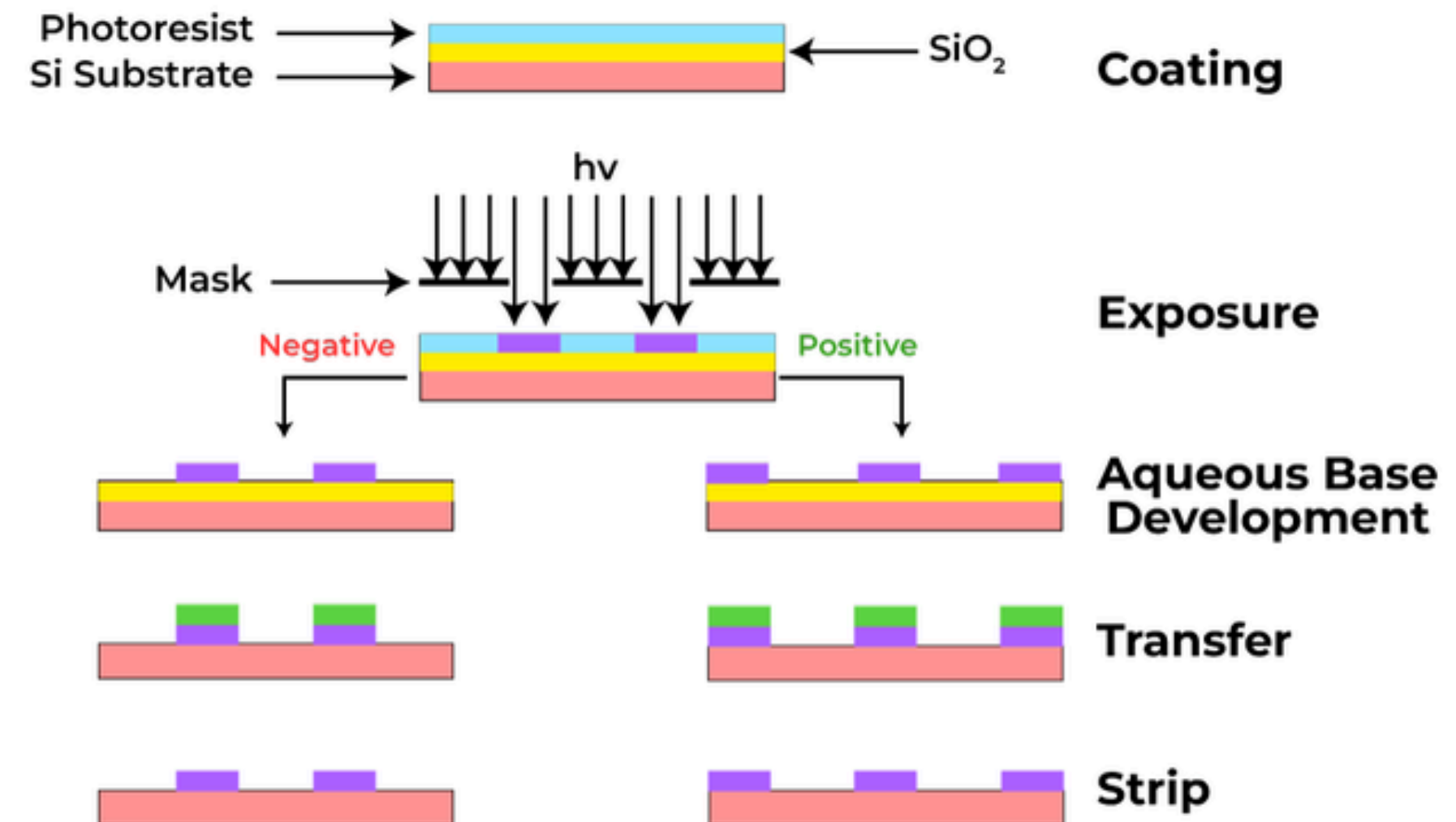
Figure 1. Comparison of model size, speed, and performance (top-1 accuracy on ImageNet-1K). Latency is profiled by iPhone 12 (iOS 16). The area of each circle is proportional to the number of parameters (model size). EfficientFormerV2 achieves high performance with small model sizes and fast inference speed.

PRODUCING GPUS

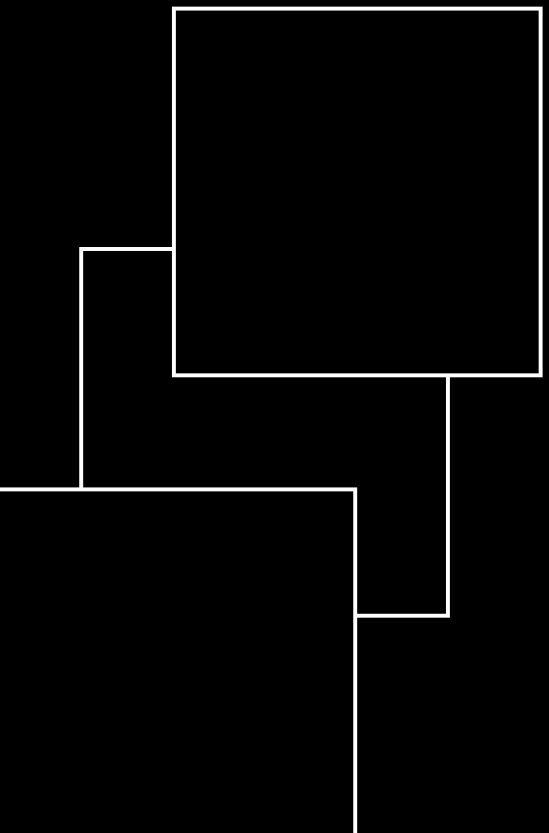
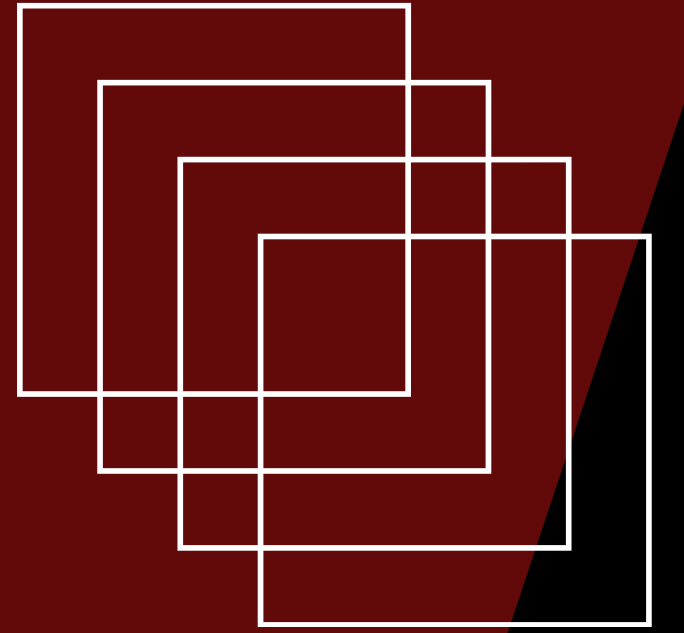
- Photolithography and ion implantation is hard
- ASML is bombarded with orders
- Many doping profiles



Photolithographic Process



THANK YOU



BIBLIOGRAPHY

[https://docs.google.com/document/
d/1xZx5r4AZp19t7UFCRz2QDtGUs3WQfgImnEgeDwD90
MY/edit?usp=sharing](https://docs.google.com/document/d/1xZx5r4AZp19t7UFCRz2QDtGUs3WQfgImnEgeDwD90MY/edit?usp=sharing)

like 20 papers lol

